Object Oriented Programming

Mihai Dascălu

Types of Classes

- Top level classes
  - Declared inside package
  - Visible throughout package, perhaps further
  - Normally, although not always, declared in their own file
    - public classes must be defined in their own file
- Nested and inner classes
  - It is possible (and useful) to define a class inside another class
  - Declared inside class (or method)
  - Can be visible only to outer class, or have wider visibility
  - The usual access modifiers (public, protected, private) can be used

Examples

- Inner classes
  - Simple and useful
- Static nested classes
- Anonymous classes
  - Useful, but syntax is rather ugly
- Method local inner classes

Inner classes

Inner Classes

- Description
  - Class defined in scope of another class
- Property
  - Can directly access all variables & methods of enclosing class (including private fields & methods)
- Example

```
public class OuterClass {
    public class InnerClass {
        ...
    }
}
```

Inner Classes

- May be named or anonymous
- Useful for:
  - Logical grouping of functionality
  - Data hiding
  - Linkage to outer class
- Examples presented in detail later on:
  - Iterator for Java Collections
  - ActionListener for Java GUI widgets

Simple Uses of Inner Classes (1)

- An inner class definition = member of the outer class equivalent to variables and methods
  - An inner class is **local** to the outer class definition
  - The name of an inner class may be reused for something else outside the outer class definition
  - If the inner class is private, then the inner class cannot be accessed by name outside the definition of the outer class

Simple Uses of Inner Classes (2)

- Two main advantages:
  - They can make the outer class **more self-contained** since inner classes are defined inside a class
  - Both classes' methods **have access to each other's private methods and instance variables**
- Helping class = one of the most useful applications of inner classes
  - The inner class should be marked private

Inner and Outer Classes Have Access to Each Other's Private Members

- Within methods of the **inner** class:
  - Reference a private instance variable of the outer class
  - Invoke a private method of the outer class
  - Essentially, the *inner class has a hidden reference to the outer class*
- Within methods of the **outer** class:
  - Reference a private instance variable of the inner class on an *object* of the inner class
  - Invoke a (nonstatic) method of the inner class if an *object* of the inner class is used as a calling object
- Within the definition of the inner or outer classes, the modifiers **`public`** and **`private`** are equivalent

Inner Class Example

- MyList

```
public class MyList {
    private Object [ ] a;
    private int size;
}
```

- Want to make MyList implement Iterable
  - Skipping generic types for now
  - Need to return an Iterator

MyIterator Design (1)

```
public class MyIterator implements Iterator {
    private MyList list;
    private int pos;
    MyIterator(MyList list) {
        this.list = list;
        pos = 0;
    }
    public boolean hasNext() {
        return pos < list.size;
    }
    public Object next() {
        return list.a[pos++];
    }
    //remove method is optional
}
```

MyIterator Design (2)

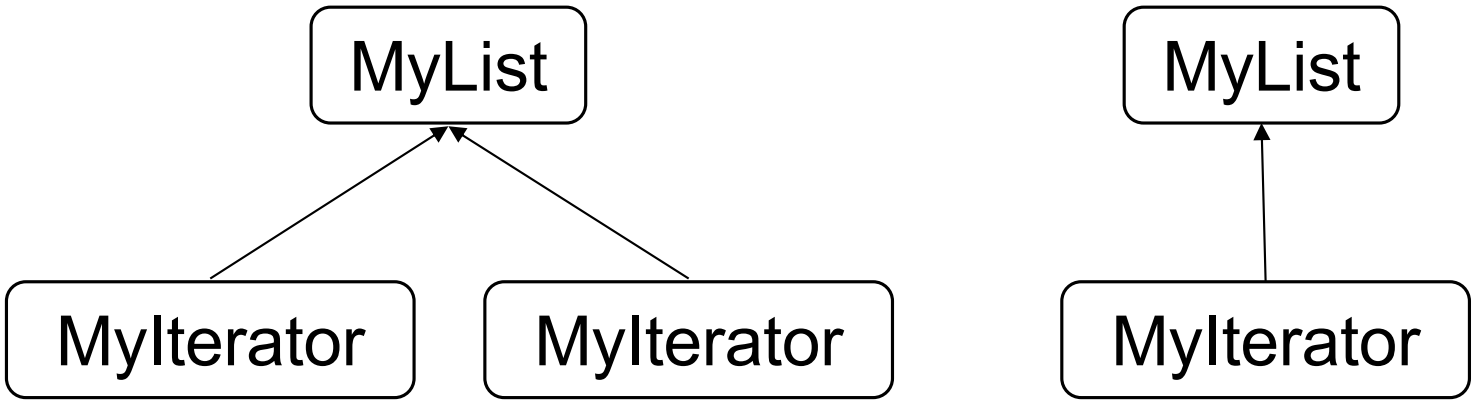
- Problems
  - Need to maintain reference to MyList
  - Need to access private data in MyList
- Solution
  - Define MyIterator as inner class for MyList

MyIterator Design (3)

```
public class MyList implements Iterable {
    private Object [ ] a;
    private int size;
    public Iterator iterator() {
     return new MyIterator();
    }
    private class MyIterator implements Iterator{
      private int pos = 0;
      public boolean hasNext()
          { return pos < size; }
      public Object next()
          { return a[pos++]; }
    }
}
```

Inner Classes

- Inner class instance
  - Has association to an instance of the outer class
  - Must be **instantiated** with an **enclosing** instance
  - Becomes tied to outer class object at moment of creation (can not be changed)



DataStructure

Referring to a Method of the Outer Class

- If a method is invoked in an inner class:
  - If the inner class **has no such method**, then
    - Invocation of the method of that name in the outer class
  - If **both have** a **method with the same name**, then
    - Invocation of the method in the inner class
  - If both have a method with the same name, & intent is to invoke the method in outer, then

```
OuterClassName.this.methodName()
```

Method resolution

- When resolving a method call on an unspecified object
  - First see if the method can be resolved on the inner object
  - If not, see if the method can be resolved on the corresponding instance of the outer object
  - If nested multiple levels, keep on looking

Creating/Referring to Inner Classes

- Assume class A defines an inner class B
- **Inside instance methods of A**, just use B as the type of references to the inner class and use new B(…) to create instances
  - Newly created B object associated with A object referenced by this
- **Outside of A**, use **A.B** to name the inner class
- If you need to create an instance of B associated with a specific object **A a**, outside of an instance method on a
  - Use a.new B()
  - Very rare, should be avoided

Public Inner Classes

- If an inner class is marked **`public`**, then
  - It can be used outside of the outer class
- In the case of a nonstatic inner class:
  - It must be **created** using an **object** of the outer class
- In case of a **static** inner class, the procedure is similar, but simpler than, that for nonstatic inner classes:

```
OuterClass.InnerClass innerObject =
                    new OuterClass.InnerClass();
```

  - Note that this is acceptable:

```
innerObject.nonstaticMethod();
```

Accessing Outer Scope Example

```
public class OC {// outer class
    int x = 2;
    public class IC {// inner class
        int x = 6;
        public void getX() {
          // inner class method
          int x = 8;
          System.out.println(x);
          System.out.println(this.x);
          System.out.println(OC.this.x);
        }
    }
}
```

Static Nested Classes

Static Nested Classes (1)

- A *normal inner* class has a connection between its objects and the outer class object that created the inner class object
  - An inner class definition can
    - Reference an instance variable
    - Invoke a method of the outer class
- There are certain situations when an inner class must be static
  - If an **object of the inner class** is **created** within a **static method** of the outer class
  - If the inner class **must have static members**

Static Nested Classes (2)

- Declared like a standard inner class, except “static class” instead of “class”
- A static method does not have access to the instance variables and nonstatic methods
- A static nested class does not have access to the instance variables and nonstatic methods of the outer class =>
  - A static member class can access only static variables of the outer class
- Not “really” an inner class, but a top-level class that happens to be written inside another class
- The actual use of static member classes is debatable

Static Nested Classes (3)

- Since a static inner class has no connection to an object of the outer class, within an inner class method
  - Instance variables of the outer class cannot be referenced
  - Non-static methods of the outer class cannot be invoked
- Preface with the name of the inner class + "." to invoke a static method or static variable
- Usage:
  - Inner object is associated with different outer objects
  - Survive longer than the outer object
  - Avoid the overhead of the extra reference in each instance of the inner object

Static Nested Class Example

```
class BigOuter {
  static class Nest {
    void go() { System.out.println("hi"); } }
  }
}
class Broom {
  static class B2 {
    void goB2() { System.out.println("hi 2"); }
  }
  public static void main(String[] args) {
    BigOuter.Nest n = new BigOuter.Nest(); // both class names
    n.go();
    B2 b2 = new B2(); // access the enclosed class
    b2.goB2();
  }
}
```

StaticExample

Anonymous Inner Classes

Anonymous Inner Class

- No name for the class
- Convenient for short code (typically just a call to some method)
- Inner class defined where you create an instance of it (in the middle of a method)
  - Useful if the only thing you want to do with an inner class is create instances of it in one location
- In addition to referring to fields/methods of the outer class, can *refer to final local variables*

Syntax for anonymous inner classes

```
new Foo() {

  public int one() { return 1; }

  public int add(int x, int y)

    {return x+y;}
};
```

- To define an anonymous inner class that:
  - Extends class Foo
  - Defines methods one and add

MyList without Anonymous Inner Class

```
public class MyList implements Iterable {
    private Object [ ] a;
    private int size;
    public Iterator iterator() {
       return new MyIterator();
     }
    public class MyIterator implements Iterator {
        private int pos = 0;
        public boolean hasNext()
          { return pos < size; }
        public Object next()
            { return a[pos++]; }
    }
}
```

MyList with Anonymous Inner Class

```
public class MyList implements Iterable {
    private Object [ ] a;
    private int size;
    public Iterator iterator() {
       return new Iterator () {
         private int pos = 0;
         public boolean hasNext()
            { return pos < size; }
         public Object next()
            { return a[pos++]; }
        };
    }
}
```

Interface Specific Anonymous inner classes

- Anonymous inner classes are
  `b.addActionListener(`***anonymous inner class***`);`
- The *anonymous inner class* can be either:

  `new` ***Superclass*****`(`*****args*****`) { `*****body*****` }`**

  or

  `new` ***Interface*****`() { `*****body*****` }`**
- No class name is given, only the name of the superclass or interface
- The *args* are arguments to the superclass's constructor (interfaces do not have constructors)

Example Anonymous Inner Class

- An ActionListener is a Java-supplied interface for listening to Buttons and some other things

```
void addActionListener(ActionListener l)
```

- The format (from the previous slide) is

```
new Interface () { body }
```

```
b.addActionListener  (new ActionListener( ) {
    public void actionPerformed (ActionEvent e) {
          System.out.println ("BAU!");
    }});
```

- Similar to member classes, anonymous inner classes have full access to the fields and methods of the containing class

Anonymous Classes Wrap-up

- If an object is to be created, but there is no need to name the object's class => anonymous class
  - Class definition embedded inside the expression with the new operator
  - Abbreviated notation for creating a simple local “in-line” object
- Sometimes used when assigned to a variable of another type (usually an interface)
- Not every inner class should be anonymous, but very simple "one-shot" local objects are such a common case

HelloWorldAnonymousClasses

Method Local Inner Classes

Method Local Inner Classes

- Defined inside a method
  - The class *declaration* is available only within that method
  - Objects created from that local class can "escape" by being assigned to nonlocal variables
- Code in the **local class cannot access variables declared in the method** unless they are declared **final (or effectively final)**
- A local class *declared* in a *static method* is subject to the same restrictions as the static method:
  - *No access to instance variables*
  - *Access to only static members* of the enclosing class (no associated instance)

Method Local Inner Class Example

```
class MyOuter2 {
  private String x = "Outer2";
  void doStuff() {
    final String z = "local variable";
    class MyInner {
      public void seeOuter() {
        System.out.println("Outer x is " + x);
        System.out.println("Local variable z is " + z);
      } // close inner class method
    } // close inner class definition
  } // close outer class method doStuff()
} // close outer class
```

LocalClassExample

Inner Classes Overview

The .class File for an Inner Class

- Compiling any class in Java produces a .class file named ClassName.class
- Inner classes compile to files with a $ in their names
- Compiling a class with one (or more) inner classes causes both (or more) classes to be compiled, and produces two (or more) .class files
  - ClassName.class and ClassName$InnerClassName.class

Multiple Inner Classes

- A class can have as many inner classes as it needs
- Inner classes have access to each other's private members if an object of the other inner class is used as the calling object

Nesting Inner Classes

- It is legal to nest inner classes within inner classes
  - The rules are the same as before, but the names become longer
  - Given class **`A`**, which has public inner class **`B`**, which has public inner class **`C`**, then the following is valid:

```
A aObject = new A();
A.B bObject = aObject.new B();
A.B.C cObject = bObject.new C();
```

Inner Classes and Inheritance

- Given an OuterClass that has an InnerClass
  - Any DerivedClass of OuterClass will automatically have InnerClass as an inner class
  - In this case, the DerivedClass cannot override the InnerClass
- An outer class can be a derived class
- An inner class can be also a derived class

Summary (1) - Inner classes (non-static nested classes)

- An ordinary class, just defined within another
- **Full access** to the variables/methods of the enclosing class (including private)
- To instantiate an inner class, you must **have** a **reference** to an instance of the outer class
  - From code within the enclosing class:
    - MyInner mi = new MyInner();
  - From code outside the enclosing class's instance methods:
    - MyOuter mo = new MyOuter();
    - MyOuter.MyInner inner = mo.new MyInner();
- From code within the inner class, *this* = reference to the inner class instance
- To reference the outer this, precede with the outer class name

Summary (2) - Static Nested Classes

- Inner classes marked with the static modifier
- It's not an inner class, it's a **top-level nested class**!
- It does not share any special relationship with an instance of the outer class
- You don't need an instance of the outer class to instantiate a static nested class
- Instantiating a static nested class requires using both the outer and nested class names:
  - BigOuter.Nested n = new BigOuter.Nested();
- Cannot access non-static members of the outer class (no outer instance) => the nested class instance does not get an outer *this* reference.

Summary (3) - Anonymous inner classes

- No name & their type must be either a subclass of the named type or an implementation of the named interface
- Useful for short definitions used in only one place
- Full access to the variables of the enclosing class
- Can extend one subclass or implement one interface (**not both**)
- Always created as part of a statement
  - Do not forget "};" !
- An *argument-defined* inner class is declared, defined and automatically instantiated as part of a method invocation – "});"

Summary (4) - Method-Local Inner Classes

- Defined within a method of the enclosing class
- For the inner class to be used, you must instantiate it & that instantiation must happen within the same method, but *after the class definition code*
- A method-local inner class cannot use variables declared within the method (including parameters) unless those variables are marked *final*
- The only modifiers you can apply to a method-local inner class are *abstract* and *final* (**Never both at the same time**)

When to use each case?

- **Non-static nested class** (**inner class**): require access to an enclosing instance's non-public fields and methods
- **Local inner class**: create more than one instance of a class, access its constructor, or introduce a new, named type *within a method*
- **Anonymous class**: declare fields or additional methods
- **Static nested class**: want to make the type more widely available & do not require access to local variables or method parameters

X